

1. INTRODUÇÃO

- **Algoritmos** são o cerne da computação. Um programa codifica um algoritmo de modo a ser executado em um computador, resolvendo assim um problema.
- É fundamental que um **programa** produza a solução com dispêndio de tempo e de memória razoáveis. Daí se deve a importância de projeto e análise de algoritmos.
- Assim, este capítulo introdutório procura ressaltar a importância de **complexidade** e de métodos de projeto e análise de algoritmos.

Visão geral do capítulo

- 1.1 A complexidade no desempenho de algoritmos.

Ilustra a importância da complexidade, situando seu impacto nos problemas que podem ser resolvidos em tempo razoável.

- 1.2 A complexidade de problemas e algoritmos.

Apresenta e motiva os principais tópicos a serem tratados.

- 1.3 Revisão matemática.

Revê algumas fórmulas matemáticas úteis no cálculo de complexidade de algoritmos.

1.1 A COMPLEXIDADE NO DESEMPENHO DE ALGORITMOS

- Um **algoritmo** é um procedimento, consistindo de um conjunto de regras não ambíguas, as quais especificam, para cada entrada, uma **seqüência finita de operações**, terminando com uma saída correspondente.
- Um algoritmo resolve um problema quando, para qualquer entrada, produz uma **resposta correta**, se forem concedidos **tempo e memória suficientes** para sua execução.
- Os recursos de **espaço** e **tempo** requeridos têm grande importância em **casos práticos**.
- Às vezes, o algoritmo mais imediato está longe de ser razoável em termos de **eficiência**.

- A tabela 1.1.1 mostra o **desempenho** desses dois algoritmos que calculam o determinante de uma matriz $n \times n$, considerando tempos de operações de um computador real.

n	Método de Cramer	Método de Gauss
2	22 μ s	50 μ s
3	102 μ s	159 μ s
4	456 μ s	353 μ s
5	2,35 ms	666 μ s
10	1,19 min	4,95 ms
20	15 225 séculos	38,63 ms
40	$5 \cdot 10^{33}$ séculos	0,315 s

Tabela 1.1.1 – Tamanho do Problema x Tempo de Execução

- O crescente avanço tecnológico, permitindo a criação de máquinas cada vez mais rápidas, pode parecer ofuscar a importância da **complexidade de tempo** de um algoritmo.

- Para ilustrar, considera-se o impacto de um **aumento de velocidade** de operação sobre alguns algoritmos, considerando uma segunda máquina dez vezes mais rápida que a primeira.
- Dados algoritmos com complexidades $\log_2 n$, n , $n \cdot \log_2 n$, n^2 , n^3 , 2^n , 3^n e um tempo t , a tabela 1.1.2 dá o **tamanho máximo** de problema resolvido por cada algoritmo no tempo t , na 1ª máquina e na 2ª dez vezes mais rápida.

Complexidade de Tempo	Tamanho máximo de problema resolvível na máquina lenta	Tamanho máximo de problema resolvível na máquina rápida
$\log_2 n$	x_0	$(x_0)^{10}$
n	x_1	$10x_1$
$n \cdot \log_2 n$	x_2	$10x_2$ (p/ x_2 grande)
n^2	x_3	$3,16x_3$
n^3	x_4	$2,15x_4$
2^n	x_5	$x_5 + 3,3$
3^n	x_6	$x_6 + 2,096$

Tabela 1.1.2 - Complexidade do algoritmo x Tamanho máximo de problema resolvível

- Supondo-se que um computador execute uma operação em $1\mu s$, chega-se à tabela 1.1.3, que descreve os **tamanhos limites** de problemas resolvíveis por algoritmos de diferentes complexidades em 1 segundo, 1 minuto e 1 hora.

Complexidade de tempo	Tamanho de problema executável em:		
	1 segundo	1 minuto	1 hora
$\log_2 n$	$2 \cdot 10^6$	$2^{6 \cdot 10^7}$	$2^{3,6 \cdot 10^9}$
n	10^6	$6 \cdot 10^7$	$3,6 \cdot 10^9$
$n \cdot \log_2 n$	62 746	$2,8 \cdot 10^6$	$1,3 \cdot 10^8$
n^2	10^3	$7,746 \cdot 10^3$	60 000
n^3	10^2	$3,9 \cdot 10^2$	$1,5 \cdot 10^3$
2^n	20	25	32
3^n	13	16	20

Tabela 1.1.3 - Complexidade do algoritmo x Tempo de execução

- As tabelas 1.1.1, 1.1.2 e 1.1.3 mostram que a **capacidade de um algoritmo**, é função de sua **complexidade**.

1.2 COMPLEXIDADE DE PROBLEMAS E ALGORITMOS

- A complexidade vem ganhando destaque a ponto de que afirmações, como a de Aho [AHU 74], de que a **complexidade** é o **coração da Computação**, já não surpreendem.

1.2.1 Metodologia de cálculo da complexidade

- A análise da complexidade de um algoritmo é realizada, usualmente, de maneira muito particular. Alguns conceitos são gerais, no sentido de que só dependem da **estrutura do algoritmo**. Esses conceitos são apresentados nos capítulos 3 e 4.

1.2.2 Complexidade na fase de projeto do algoritmo

- O estudo das **técnicas** de desenvolvimento de algoritmo permite prever alguns aspectos da complexidade do algoritmo resultante.
- O **projeto** de algoritmos eficientes é sempre um propósito de qualquer projetista e é uma preocupação já nessa fase.
- No capítulo 5, alguns **métodos** de desenvolvimento de algoritmos são apresentados, e a complexidade dos algoritmos gerados é analisada.

1.2.3 A complexidade do problema e a intratabilidade

- A complexidade também pode ser vista como uma **propriedade do problema**, o que significa dar uma medida independente do tratamento dado ao problema. Alguns problemas são “bem comportados” e permitem chegar a limites de complexidade.
- Outros problemas parecem ficar tão **difíceis** de serem resolvidos para instâncias grandes que parecem tornar-se **intratáveis**. Esses problemas são apresentados no capítulo 6.

1.3 REVISÃO MATEMÁTICA

- No **cálculo da complexidade** de um algoritmo, é necessário o uso de conceitos e resultados matemáticos.

- Soma dos termos de uma progressão aritmética:

$$\sum_{i=0}^n a_i = \frac{(a_0 + a_n) \cdot (n+1)}{2} \quad \text{com } a_i = a_0 + i \cdot n.$$

- Soma dos termos de uma progressão geométrica:

$$\sum_{i=0}^{n-1} a \cdot q^i = \frac{a \cdot (q^n - 1)}{q - 1} \quad \text{para } q \neq 1.$$

- Binômio de Newton:

$$(a + b)^n = \sum_{i=0}^n \binom{n}{i} a^{n-i} \cdot b^i$$

- Outras somas:

$$\sum_{i=0}^n i^2 = \frac{1}{6} \cdot n \cdot (n+1) \cdot (2n+1)$$

$$\sum_{i=0}^n i \cdot 2^i = 2 + (n-1) \cdot 2^{n+1}$$

- Combinação:

$$\binom{n}{k} = \frac{n!}{(n-k)! \cdot k!}$$

- Somas combinatoriais também são comuns em cálculos de complexidade:

$$\sum_{i=0}^n \binom{n}{k} \cdot (x-1)^k = x^n$$

- Logaritmos também são funções comuns no estudo da complexidade. Por definição,

$$a^{\log_a n} = n$$

- Propriedades:

$$\log_a (n \cdot m) = \log_a n + \log_a m$$

$$\log_a \left(\frac{n}{m}\right) = \log_a n - \log_a m$$

$$\log_a (n^m) = m \cdot \log_a n$$

$$a^{\log_c b} = b^{\log_c a}$$

$$\log_a n = \log_b n + \log_a b$$

1.4 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é ressaltar a importância da complexidade e de métodos de projeto e análise de algoritmos.
- Nem sempre o algoritmo mais imediato é viável na prática.
- Algoritmos mais eficientes costumam tirar maior proveito de máquinas melhores.
- A análise de complexidade de um algoritmo pode ser feita com base em sua estrutura. Há métodos gerais de desenvolvimento de algoritmos com amplo espectro de aplicação.
- A complexidade de um problema é a do melhor algoritmo imaginável (conhecido ou não) que possa resolvê-lo.
- Fórmulas matemáticas envolvendo somatórios, como na seção 3, são úteis para a análise de complexidade de algoritmos.